

Q1. Write a class demonstrating encapsulation and inheritance.

```
class Person {
    private String name; // encapsulated (hidden) field
    private int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

class Student extends Person { // inheritance
    private String course;

    Student(String name, int age, String course) {
        super(name, age);
        this.course = course;
    }

    public void displayStudentInfo() {
        displayInfo();
        System.out.println("Course: " + course);
    }
}

public class EncapsulationInheritanceDemo {
```

```

        public static void main(String[] args) {
            Student s = new Student("Ravi", 20, "Computer Science");
            s.displayStudentInfo();
        }
    }

```

Encapsulation is shown in the Person class, where name and age are declared private and can only be accessed through public getter methods. Inheritance is shown by Student extending Person, gaining access to Person's public behavior and fields (indirectly through inherited methods) while adding its own field, course.

Q2. Implement abstraction using an abstract class.

```

abstract class Appliance {
    abstract void turnOn();    // no implementation -- pure specification
    abstract void turnOff();

    void showStatus() {        // shared concrete behavior
        System.out.println("Status check complete.");
    }
}

class WashingMachine extends Appliance {
    @Override
    void turnOn() {
        System.out.println("Washing machine started.");
    }

    @Override
    void turnOff() {
        System.out.println("Washing machine stopped.");
    }
}

public class AbstractionDemo {
    public static void main(String[] args) {
        Appliance a = new WashingMachine();
        a.turnOn();
        a.showStatus();
        a.turnOff();
    }
}

```

Appliance is declared abstract and cannot be instantiated directly; it defines what any appliance must be able to do (turnOn, turnOff) without specifying how. WashingMachine provides the concrete implementation. This hides the differing internal details of each appliance type behind a common abstract interface.

Q3. Show polymorphism with method overriding in Java.

```
class Animal {
    void makeSound() {
        System.out.println("Some generic animal sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Bark");
    }
}

class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Meow");
    }
}

public class PolymorphismDemo {
    public static void main(String[] args) {
        Animal[] animals = { new Dog(), new Cat(), new Animal() };
        for (Animal a : animals) {
            a.makeSound(); // resolved at run time (dynamic dispatch)
        }
    }
}
```

Each subclass, Dog and Cat, overrides the makeSound method inherited from Animal, providing its own specific behavior. Even though all elements in the array are declared as type Animal, the actual method executed for each element is determined by its real, run-time type -- this is runtime polymorphism, achieved through method overriding.